MONISH.S

192311132

CSE

CSA4312-INTERNET PROGRAMMING

CO1-AT2

1. HTTP Response Analysis

1. Identify incorrect or invalid fields

Given response:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Errors and observations:

- Content-Length: -150 — Invalid because content length cannot be negative.
- Date — The weekday is incorrect: 1 April 2026 is a Wednesday, not Monday.
- Content-Type: text/html — Valid for an HTML response.
- Connection: keep-alive — Valid for HTTP/1.1.
- 200 OK — Indicates success, but does not guarantee that every header is valid.

2. Issue with Content-Length

`Content-Length` specifies the size of the response body in bytes.

A negative value is invalid and can cause parsing errors or unpredictable behavior in HTTP implementations. The value must be the actual body size in bytes, not an arbitrary number.

3. Does the status code match?

`200 OK` is appropriate only if the request was successfully handled. However, the invalid `Content-Length` makes the response malformed, so the status code alone does not establish that the response is correct.

4. Corrected response

Example, assuming the body is the HTML text `Hello` (5 bytes):

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 5
Connection: keep-alive

Hello
```

2. DOM Structure Analysis

1. Identify missing or improperly closed elements

Original code:

```
<div>
    <p>HelloWorld
</div>
```

The `<p>` element is missing its closing `</p>` tag. The remaining outer elements are closed.

2. How does the browser interpret it?

HTML browsers use error-recovery rules. When the browser encounters `</div>`, it automatically closes the open paragraph before closing the `div`.

The browser may therefore render the page despite the missing tag.

3. Impact on rendering and layout

- The paragraph may appear correctly in simple cases.
- Unexpected spacing or styling can occur in more complicated markup.
- JavaScript that examines or modifies the DOM may behave differently than intended.
- Invalid markup can cause accessibility and maintenance issues.

4. Corrected DOM structure

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>TestPage</title>
</head>
<body>
    <div>
        <p>HelloWorld</p>
    </div>
</body>
</html>
```

3. GET vs POST in Login System

1. Differences between GET and POST

Feature	GET	POST
Data location	URL query string	Request body
Data visibility	Visible in URL	Not normally visible in URL
Browser history	Parameters may be recorded	Body is not normally recorded in history
Common purpose	Retrieve data	Submit or process data
Login suitability	Not recommended	Preferred

Important: POST does not automatically encrypt data. HTTPS is required to protect credentials during transmission.

2. Which method is suitable for login?

POST is more suitable because usernames and passwords are sent in the request body rather than being placed in the URL.

3. Security risks of GET authentication

- Passwords may appear in browser history.
- URLs may be stored in server logs and monitoring systems.
- URLs may be shared or copied accidentally.
- Query parameters may leak through referrer information in some circumstances.
- Credentials can be exposed if the connection is not encrypted.

4. Best practices for secure login

1. Use HTTPS for all authentication requests.

2. Submit credentials using POST.
3. Validate credentials on the server.
4. Store passwords using a strong salted password-hashing algorithm.
5. Use secure session cookies.
6. Apply rate limiting and account-abuse protections.
7. Never store plain-text passwords in the browser or database.

4. HTML Code Inspection

1. Identify missing closing tags

Original snippet:

```
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
```

The `<h1>` and `<p>` elements are missing closing tags. The `<img>` element is a void element, so it does not require a closing tag.

2. Issues with the `<img>` tag

- The `alt` attribute is missing.
- If the image cannot load, users have no alternative text.
- Screen readers cannot communicate the image's meaning when meaningful alternative text is needed.

3. How do browsers handle the errors?

Browsers usually recover from missing closing tags and display the page. However, relying on automatic recovery can create unexpected DOM structures and inconsistent styling or scripting behavior.

4. Corrected HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sample</title>
</head>
<body>
    <h1>Welcome</h1>
```

```
    <p>This is a paragraph.</p>

    <img src="image.jpg" alt="Sample image">
</body>
</html>
```

5. Browser-Server Communication Flow

1. Explain each step

Step 1: User enters a URL

The browser reads the address and identifies the destination website and requested resource.

Step 2: DNS lookup

DNS translates the domain name, such as `example.com`, into an IP address so the browser can locate the server. A cached result may avoid a new DNS lookup.

Step 3: HTTP request

The browser establishes or reuses a connection and sends a request containing the method, path, headers, and possibly a body. HTTPS also provides TLS encryption.

Step 4: Server processes request

The server routes the request, performs necessary operations, and prepares the response.

Step 5: Server returns response

The response contains a status code, headers, and possibly a body containing HTML, JSON, an image, or other content.

Step 6: Browser renders the page

The browser parses HTML, constructs the DOM, applies CSS, executes JavaScript, and loads required resources to display the page.

2. Where can delays occur?

- DNS resolution.
- Establishing a TCP connection and TLS handshake.
- Network latency and slow internet.
- Server processing or database queries.
- Large HTML, CSS, JavaScript, or image files.

- Slow client-side JavaScript and rendering.

3. Role of DNS

DNS stands for Domain Name System. It translates human-readable domain names into IP addresses that computers use to communicate. DNS caching can reduce repeated lookup time.

4. Performance optimization techniques

1. Use DNS caching and a reliable DNS provider.
2. Enable HTTPS with efficient connection reuse.
3. Use a Content Delivery Network (CDN) for static resources.
4. Compress responses using Brotli or Gzip.
5. Optimize images and use modern formats such as WebP or AVIF.
6. Minify CSS and JavaScript.
7. Cache static content using suitable HTTP cache headers.
8. Optimize database queries and server-side code.
9. Reduce unnecessary HTTP requests and third-party scripts.
10. Use browser developer tools to measure loading time.

Result: HTTP response validation, DOM correction, secure form submission, proper HTML structure, and understanding of browser-server communication help develop reliable, accessible, secure, and efficient websites.